Centro Paula Souza

Faculdade de Tecnologia de Votorantim

Curso de Ciência de Dados para Negócios

Aluno: Ana Elisa Rubinato Cavalheiro

Técnicas de Programação para Ciência de Dados

Roteiro de Perguntas – Estruturas de Dados em Python 2.

Dicionários em Python

• O que acontece se você tentar acessar uma chave que não existe?

Se você tentar acessar uma chave inexistente usando a sintaxe de colchetes (meu_dicionario['chave_inexistente']), o Python levantará um erro do tipo KeyError.

• É possível adicionar uma nova chave ao dicionário depois de criado?

Sim, é totalmente possível. Dicionários são mutáveis. Você pode adicionar um novo par chave-valor simplesmente atribuindo um valor a uma nova chave: meu_dicionario['nova_chave'] = novo_valor. Se a chave já existir, essa operação atualizará o valor associado a ela.

Tuplas

• O que acontece se você tentar alterar um valor de uma tupla?

Tentar alterar um elemento de uma tupla após sua criação resultará em um erro do tipo TypeError. Isso ocorre porque as tuplas são imutáveis, ou seja, seu conteúdo não pode ser modificado depois de criadas.

• Quando seria melhor usar uma tupla em vez de uma lista?

Imutabilidade: Quando você quer garantir que os dados não sejam alterados acidentalmente após a criação (ex: coordenadas (x, y), cores RGB (r, g, b)).

Chaves de dicionário: Como são imutáveis, tuplas podem ser usadas como chaves em dicionários (listas não podem).

Retorno de funções: Para retornar múltiplos valores de uma função de forma organizada.

Heterogeneidade fixa: Quando você tem uma coleção de itens de tipos diferentes, mas com uma estrutura fixa (ex: (nome, idade, cidade)).

Performance (marginal): Em alguns casos, tuplas podem ser ligeiramente mais eficientes em termos de memória e velocidade de iteração do que listas, mas essa

diferença geralmente é pequena. A principal razão para usar tuplas é a imutabilidade semântica.

• **Como se acessa os elementos de uma tupla?**

Os elementos de uma tupla são acessados usando índices numéricos (começando do zero), da mesma forma que nas listas. Exemplo: minha_tupla[0] acessa o primeiro elemento, minha_tupla[1] o segundo, e assim por diante. Também suportam índices negativos (minha_tupla[-1] para o último elemento).

Arrays NumPy

• **Qual é o tipo de dados armazenado em um array? Como verificar isso?**

Um array NumPy armazena elementos de um único tipo de dados (homogêneo), como inteiros (int64), números de ponto flutuante (float64), booleanos (bool), etc. Isso é crucial para a eficiência do NumPy. Você verifica o tipo de dados usando o atributo. dtype do array. Exemplo: meu_array.dtype.

• **Todos os elementos têm o mesmo tipo?**

Sim, essa é a característica principal dos arrays NumPy para otimização de performance. Todos os elementos em um array NumPy padrão *devem* ter o mesmo tipo de dados (dtype). Existe a possibilidade de criar arrays com dtype=object, que podem conter tipos diferentes, mas isso perde a maioria das vantagens de performance do NumPy.

• **Qual a diferença entre arange() e linspace()?**

numpy.arange(start, stop, step): Cria um array com valores espaçados uniformemente definidos por um step (passo), dentro de um intervalo [start, stop). O valor stop não é incluído. É similar ao range() do Python, mas retorna um array NumPy.

numpy.linspace(start, stop, num): Cria um array com um número específico (num) de elementos, espaçados uniformemente entre start e stop. Por padrão, o valor stop é incluído no array resultante.

• **Para que serve a função reshape()?**

A função numpy.reshape() (ou o método. reshape() de um array) serve para mudar a forma (dimensões) de um array sem alterar seus dados. Por exemplo, você pode transformar um array 1D de 12 elementos em um array 2D de 3x4 ou 4x3. Ela retorna um novo array com a nova forma, que geralmente é uma *view* (compartilha os dados com o original) se possível. O número total de elementos deve permanecer o mesmo.

Performance: lista vs array

• **Qual estrutura foi mais rápida? Por quê?**

NumPy arrays são significativamente mais rápidos para operações numéricas (matemáticas, lógicas, etc.) aplicadas a todos os elementos.

Tipagem Homogênea e Memória Contígua: Arrays NumPy armazenam elementos do mesmo tipo em blocos contíguos de memória. Listas Python armazenam

ponteiros para objetos que podem estar espalhados pela memória e podem ser de tipos diferentes.

Implementação em C/Fortran: As operações do NumPy (ufuncs) são implementadas em C ou Fortran, que são linguagens compiladas e muito mais rápidas que o Python interpretado para loops e operações matemáticas.

Vectorization (Vetorização): NumPy permite operações vetorizadas, onde uma única instrução opera sobre múltiplos elementos do array de uma vez (usando otimizações como SIMD - Single Instruction, Multiple Data), eliminando a necessidade de loops explícitos em Python, que são lentos.

Operadores e broadcasting

- **O que acontece se os arrays tiverem tamanhos diferentes?**

NumPy tenta aplicar o broadcasting. Se as formas dos arrays forem compatíveis segundo as regras de broadcasting (as dimensões são comparadas da direita para a esquerda e devem ser iguais, ou uma delas deve ser 1, ou uma delas não existir), o array menor é "esticado" virtualmente para se ajustar à forma do maior, e a operação é realizada. Se as formas não forem compatíveis para broadcasting, NumPy levantará um erro ValueError.

- **A operação foi feita elemento a elemento? Como você sabe?**

Sim, as operações aritméticas padrão (+, -, *, /, **, etc.) e lógicas entre arrays NumPy (ou entre um array e um escalar) são realizadas elemento a elemento (element-wise).

Você sabe disso porque o array resultante tem uma forma compatível (a forma do array original ou a forma resultante do broadcasting), e cada elemento do resultado é o produto da operação aplicada aos elementos correspondentes nos arrays de entrada.

ufuncs (funções universais)

- **O que a função faz exatamente?**

Uma ufunc (universal function) é uma função que opera em arrays NumPy de forma elemento a elemento. Elas são o núcleo da capacidade de computação vetorizada do NumPy. Exemplos incluem np.add, np.subtract, np.sin, np.cos, np.exp, np.sqrt, np.log, etc. Elas pegam um ou mais arrays como entrada e produzem um ou mais arrays como saída.

- **A função altera o array original ou retorna um novo?**

A maioria das ufuncs retorna um novo array com os resultados da operação. Elas não modificam os arrays de entrada originais.

É possível fazer operações *in-place* (que modificam o original) usando operadores como +=, -=, *= ou especificando o array de saída com o argumento out em algumas ufuncs (ex: np.add(a, b, out=a)).

Indexação e slicing

- **O que representa o índice negativo?**

O índice negativo (-n) representa a indexação a partir do final do array (ou lista/tupla). -1 refere-se ao último elemento, -2 ao penúltimo, e assim por diante.

• O slicing inclui o índice final?

Não. Na sintaxe de slicing [start:stop:step], o índice start é incluído, mas o índice stop é excluído. O slice vai *até*, mas não inclui, o elemento na posição stop.

• O resultado é uma cópia ou uma view?

No NumPy, o slicing básico (usando apenas a sintaxe start:stop:step) retorna uma view (vista) do array original. Isso significa que o slice é apenas uma "janela" para os dados originais; modificar o slice *também modificará* o array original. Isso é feito por eficiência, para evitar cópias desnecessárias de dados.

A indexação avançada (usando listas de índices, arrays de inteiros ou arrays booleanos) retorna uma cópia dos dados, não uma view.

View (shallow copy) vs cópia profunda

• Qual é o id() do array original e da cópia? Eles são iguais?

- id() retorna o identificador único de um objeto na memória.

- O id() do array original e de uma view (obtida por slicing, por exemplo) serão diferentes, pois são objetos Python distintos. No entanto, eles podem compartilhar o mesmo buffer de dados subjacente.

- O id() do array original e de uma cópia profunda (obtida com .copy()) serão diferentes, e eles terão buffers de dados completamente separados.

• O que acontece ao alterar a view? Isso afeta o original?

Sim. Como a view compartilha os mesmos dados subjacentes do array original, alterar um elemento na view afetará o elemento correspondente no array original, e vice-versa.

• Quando devemos usar cópia profunda?

Você deve usar uma cópia profunda (usando o método .copy()) sempre que precisar modificar o novo array sem afetar o array original. É útil para isolar os dados e evitar efeitos colaterais indesejados quando você passa arrays para funções ou quando precisa manter o estado original intacto.

Flatten vs Ravel

• Ambos retornam arrays unidimensionais? Qual a diferença real entre eles?

Sim, tanto flatten() quanto ravel() retornam uma versão unidimensional (achatada) do array.

Diferença principal:

- ndarray.flatten(): Sempre retorna uma cópia do array original, achatada. Modificar o resultado de flatten() nunca afetará o array original.
- numpy.ravel(array) ou ndarray.ravel(): Retorna uma view do array original sempre que possível (por exemplo, se o array original já estiver contíguo na memória na ordem correta). Se uma view não for possível, ele retorna uma cópia. É potencialmente mais eficiente em termos de memória se uma view puder ser retornada.

• O que acontece se você alterar um valor no array retornado por ravel()?

Se ravel() retornou uma view, alterar o array achatado modificará o array multidimensional original.

Se ravel() retornou uma cópia (porque uma view não era possível), alterar o array achatado não afetará o array original.

Transposição e empilhamento

• Qual é o formato do array após a transposição?

A transposição (usando o atributo .T ou a função numpy.transpose()) inverte ou permuta os eixos do array. Para um array 2D com forma (linhas, colunas), a transposta terá a forma (colunas, linhas). Para arrays de dimensões mais altas, ela permuta os eixos (por padrão, inverte a ordem deles).

• hstack aumenta o número de colunas ou de linhas?

numpy.hstack() (horizontal stack) empilha arrays lado a lado, na horizontal. Portanto, ele aumenta o número de colunas (o tamanho do segundo eixo, ou eixo 1 em 2D). O número de linhas deve ser o mesmo entre os arrays empilhados.

• Quais as condições para que vstack e hstack funcionem corretamente?

numpy.vstack() (vertical stack): Os arrays a serem empilhados verticalmente (um sobre o outro) devem ter o mesmo número de colunas (a mesma forma ao longo de todos os eixos, exceto o primeiro - eixo 0).

numpy.hstack() (horizontal stack): Os arrays a serem empilhados horizontalmente (lado a lado) devem ter o mesmo número de linhas (a mesma forma ao longo de todos os eixos, exceto o segundo - eixo 1). Para arrays 1D, eles são tratados como linhas antes do empilhamento horizontal, resultando num array 1D mais longo.